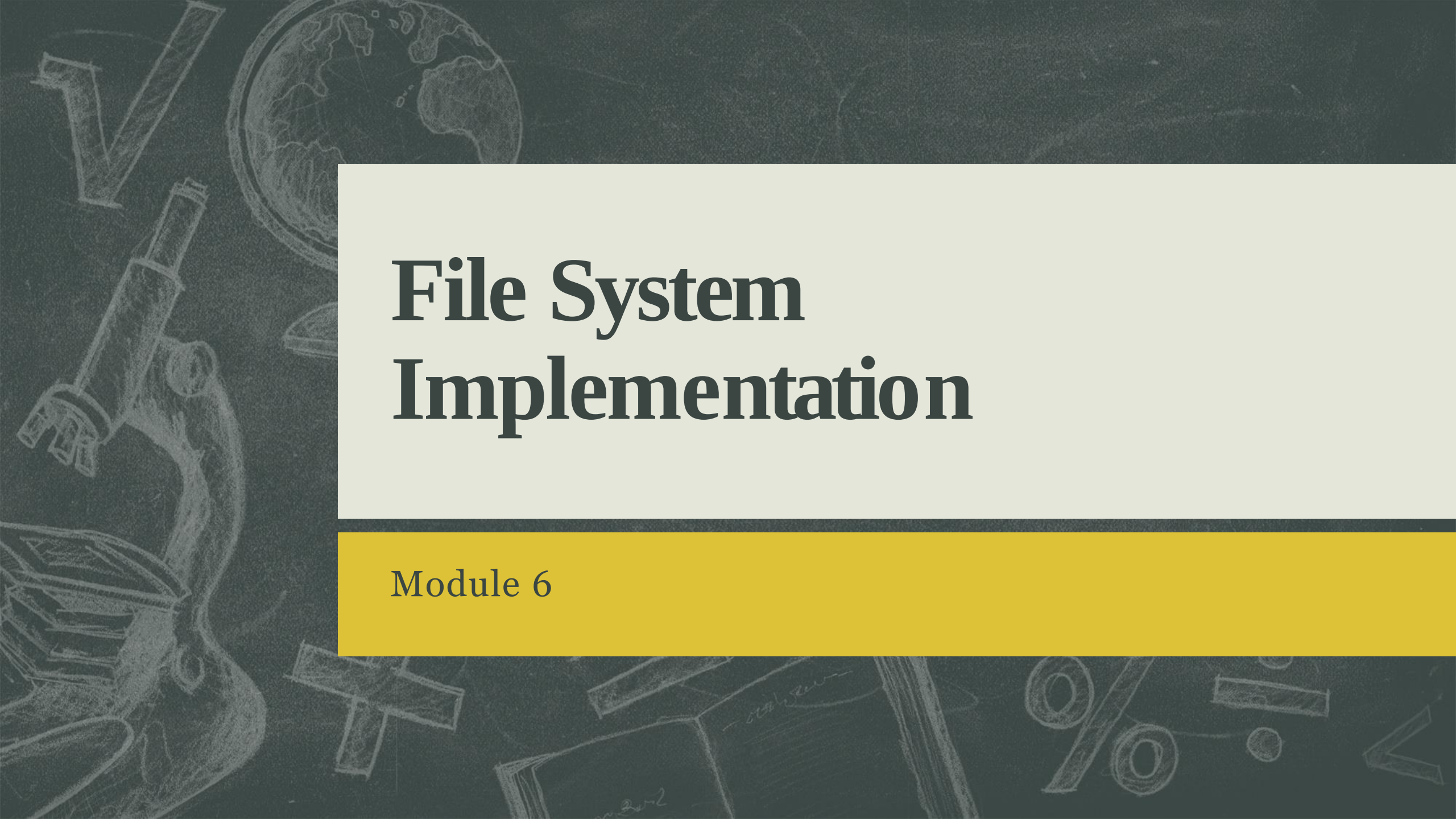




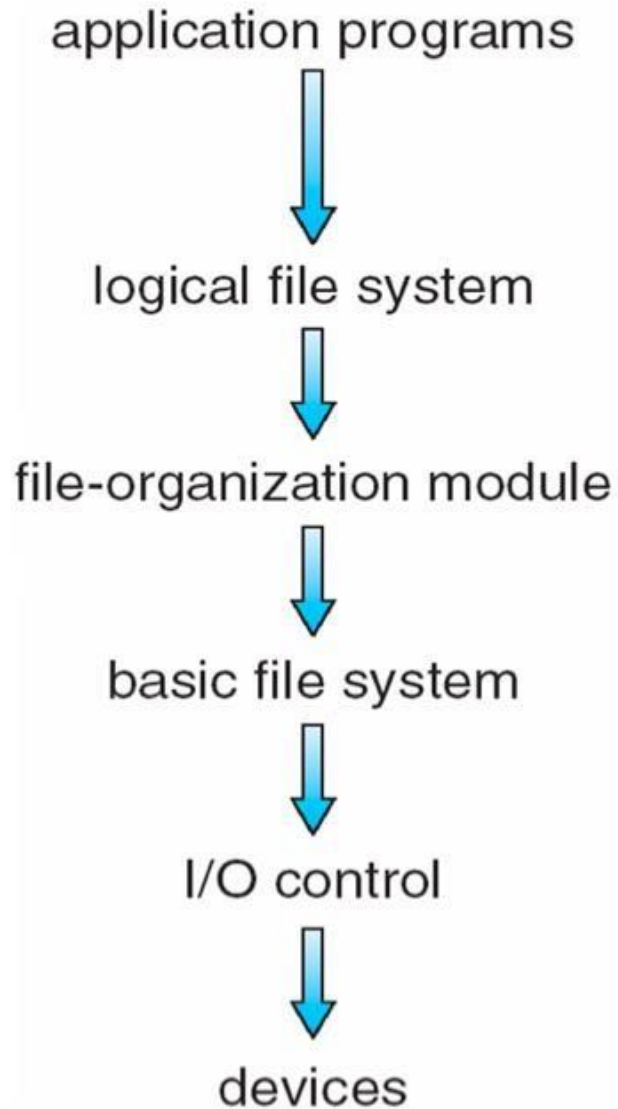
File System Implementation

Module 6

File System Implementation

- File system is usually implemented and organized into layers.
- Layering can reduce implementation complexity and redundancy.
- It may increase overhead and decrease performance.

File System Implementation – Layered File System



File System Implementation – Layered File System

■ Device Drivers

- Device drivers manage disk devices at the I/O control layer
 - Device driver accepts commands to access raw disk
 - Command “read drive1, cylinder 72, track 2, sector 10, into memory 1060”
 - It converts the command to hardware devices access (i.e., using registers)

File System Implementation – Layered File System

■ Basic File System

- Basic file system provides methods to access physical blocks
- It translates commands like “retrieve block 1 23” to device driver
- Manages memory buffers and caches (allocation, freeing, replacement)

File System Implementation – Layered File System

- **File Organization Module**

- It understands **files**, **logical address**, and **physical blocks**
- It translates **logical block #** to **physical block #**
- It manages free space, disk allocation

File System Implementation – Layered File System

- **Logical file system**

- It understand file system structures (metadata)
- It translates file name into file number, file handle, location by maintaining file control blocks (inodes in Unix)
- Directory management and protection

File System Implementation

- File-system needs to maintain **on-disk** and **in-memory structures**
 - on-disk for **data storage**, in-memory for **data access**
- **On-disk structure** has several control blocks
 - **Boot control block** contains info to boot OS from that volume
 - only needed if volume contains OS image, usually first block of volume
 - **Volume control block** (e.g., superblock) contains volume details
 - total # of blocks, # of free blocks, block size, free block pointers or array
 - **Directory structure** organizes the directories and files
 - file names and layout
 - **per-file file control block** contains many details about the file
 - inode number, permissions, size, dates

File System Implementation – A Typical File Control Block

file permissions

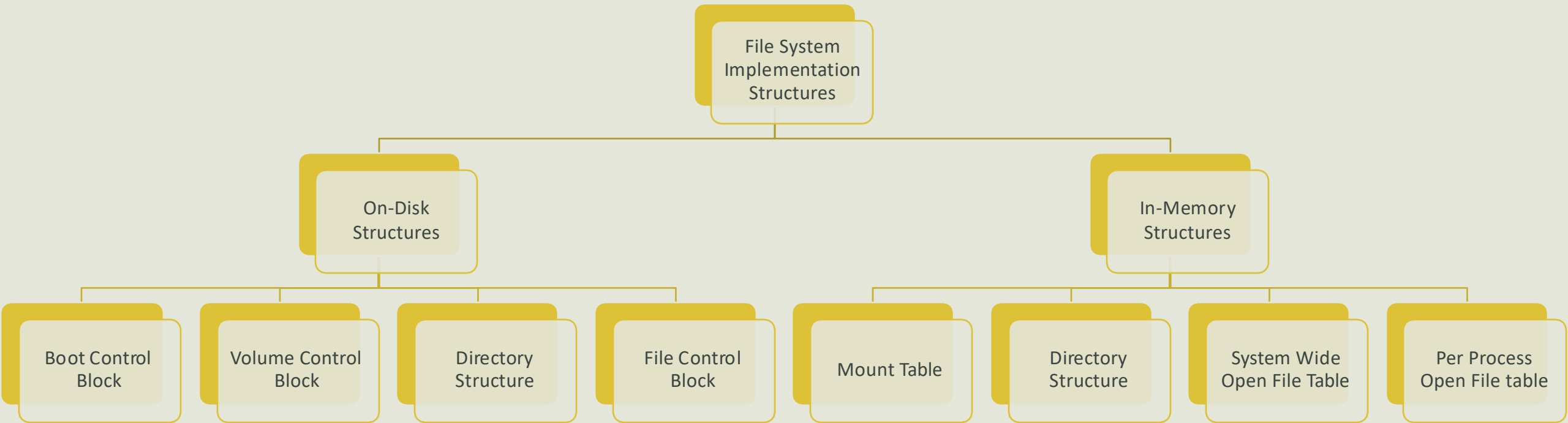
file dates (create, access, write)

file owner, group, ACL

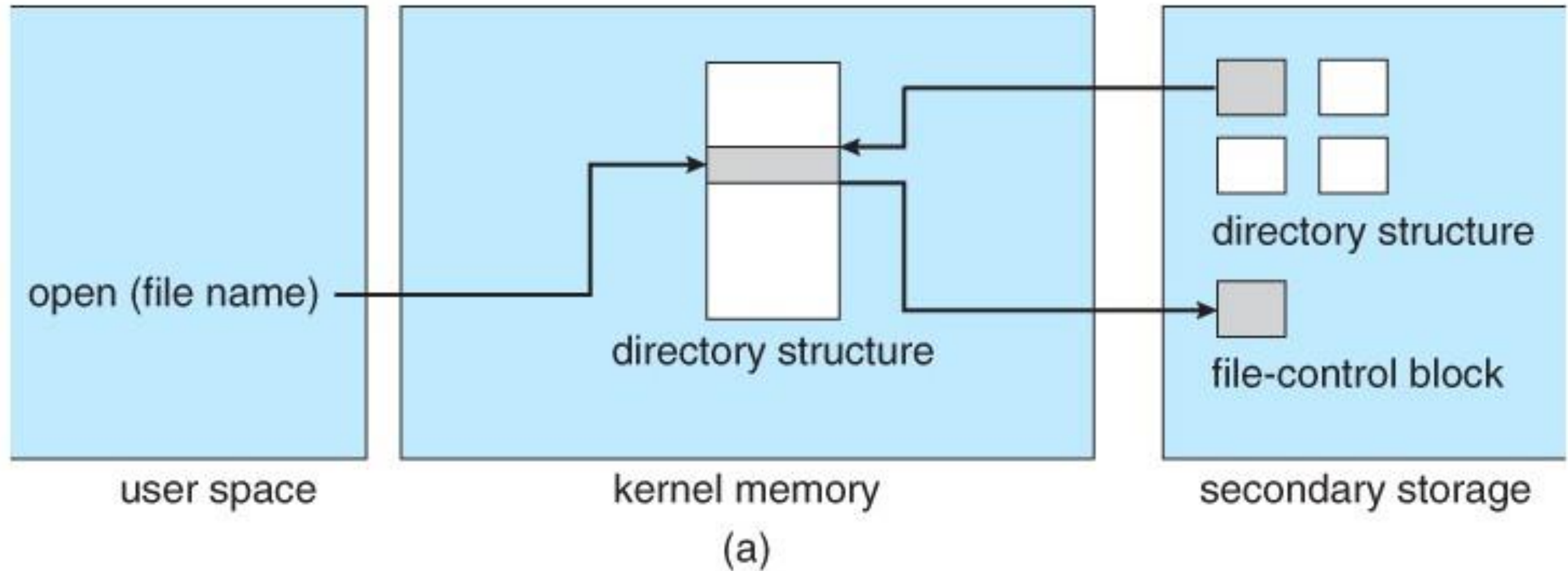
file size

file data blocks or pointers to file data blocks

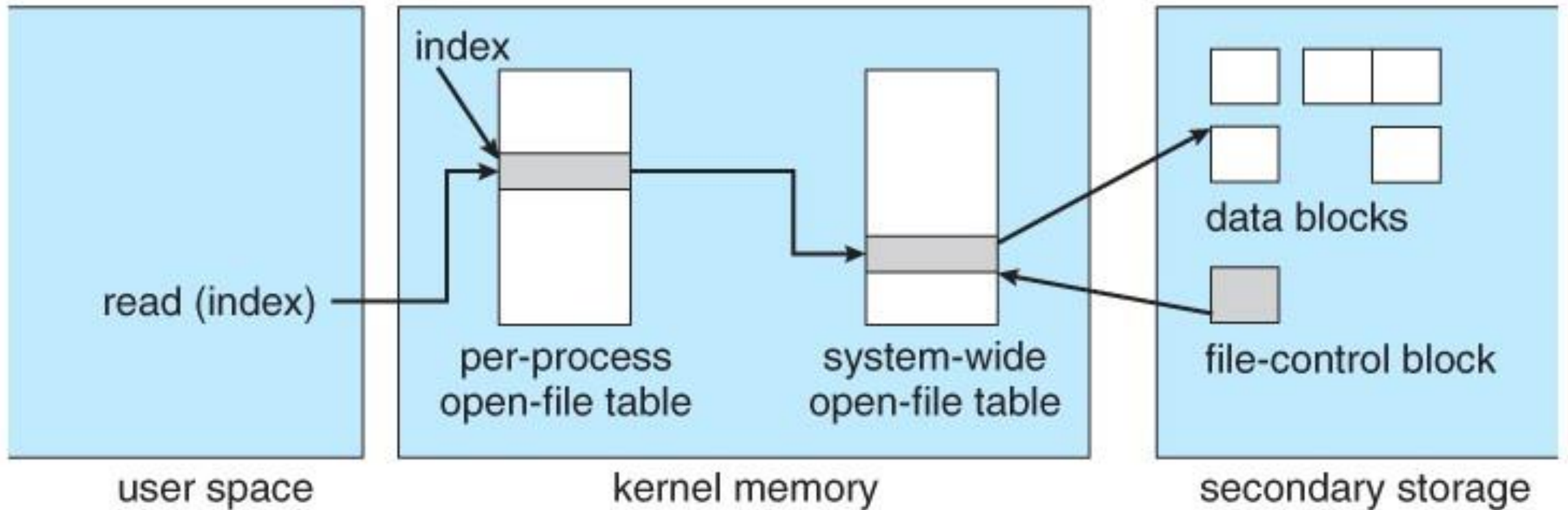
File System Implementation – Structures



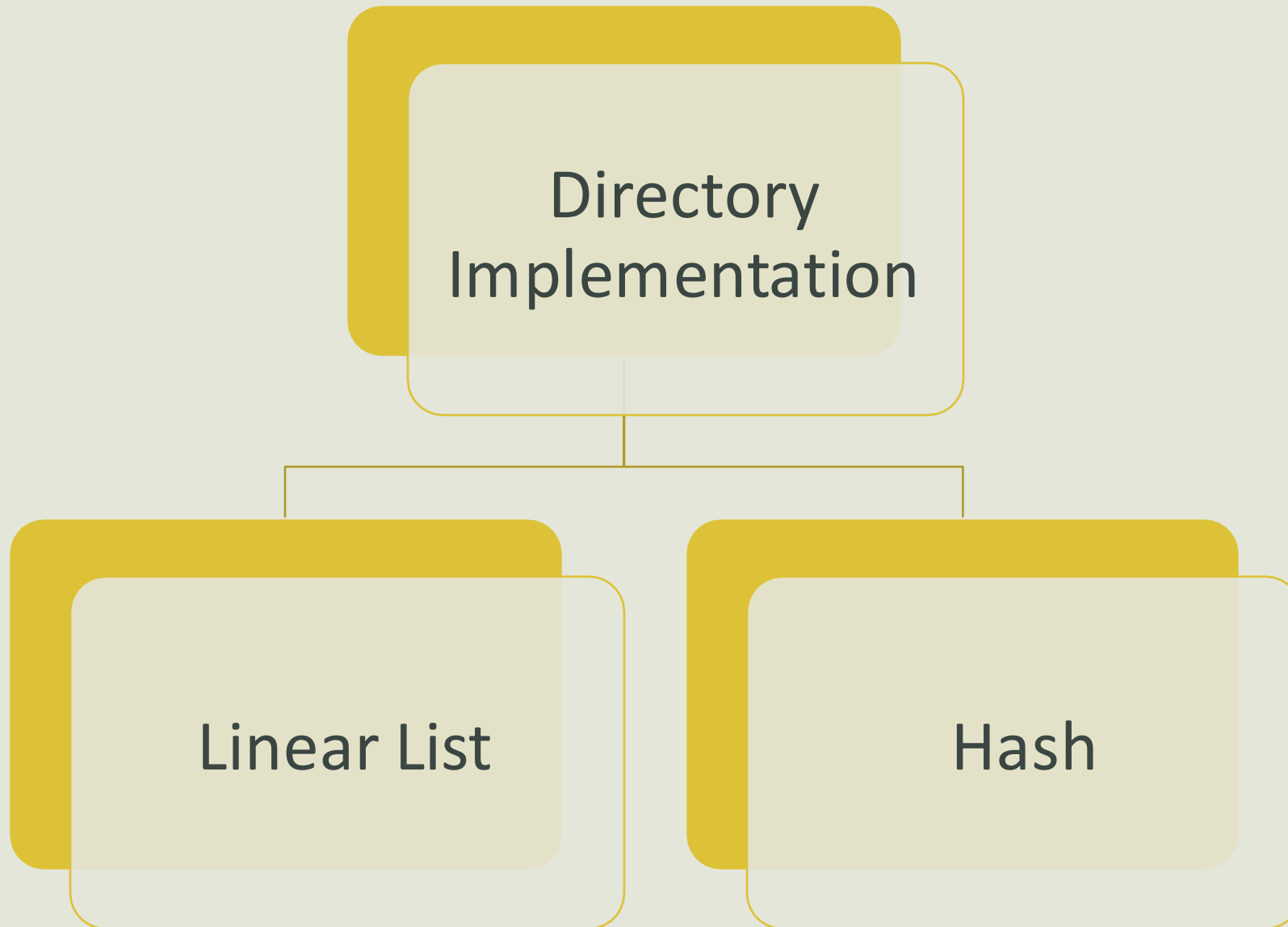
File System Implementation – Structures



File System Implementation – Structures



File System Implementation – Directory Implementation

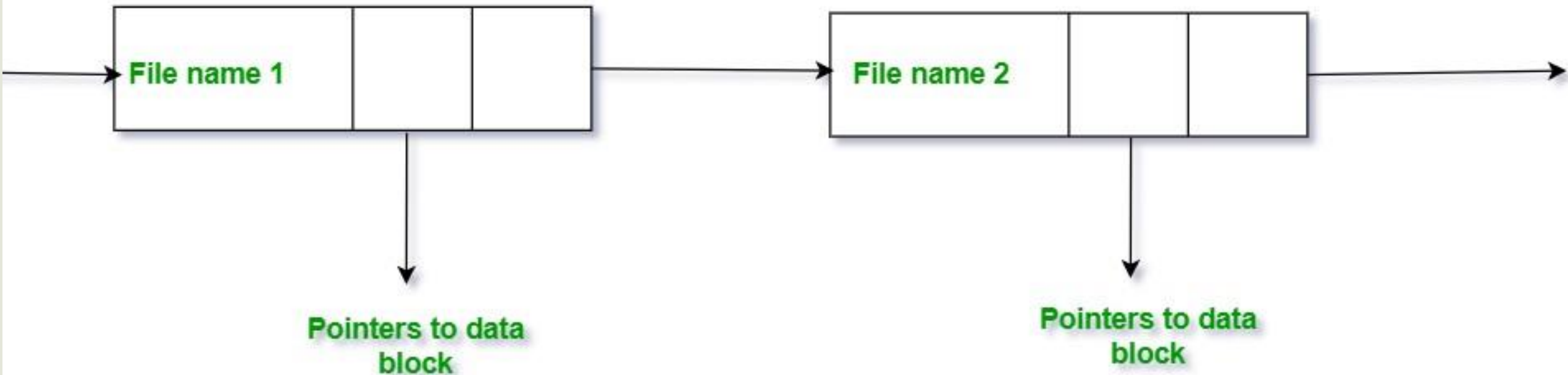


Directory Implementation – Linear List

- A **linear list of file names** is maintained **with pointers to the data blocks**
- Simple to program but time-consuming to execute
 - To **create** a new file, the directory is searched to find if another file with a similar name exists. If no such file exists, a new entry is added to the end of the directory.
 - To **delete** a file, the directory is searched for that file and the space allocated to it is released.
 - To **reuse** this deleted entry, the entry is marked as unused by assigning it a special name or it is attached to a list of free directory entries or the last entry in the directory is copied to the freed location and the length of the directory is decreased.

Directory Implementation – Linear List

Directory Implementation Using Singly Linked List



Directory Implementation – Linear List

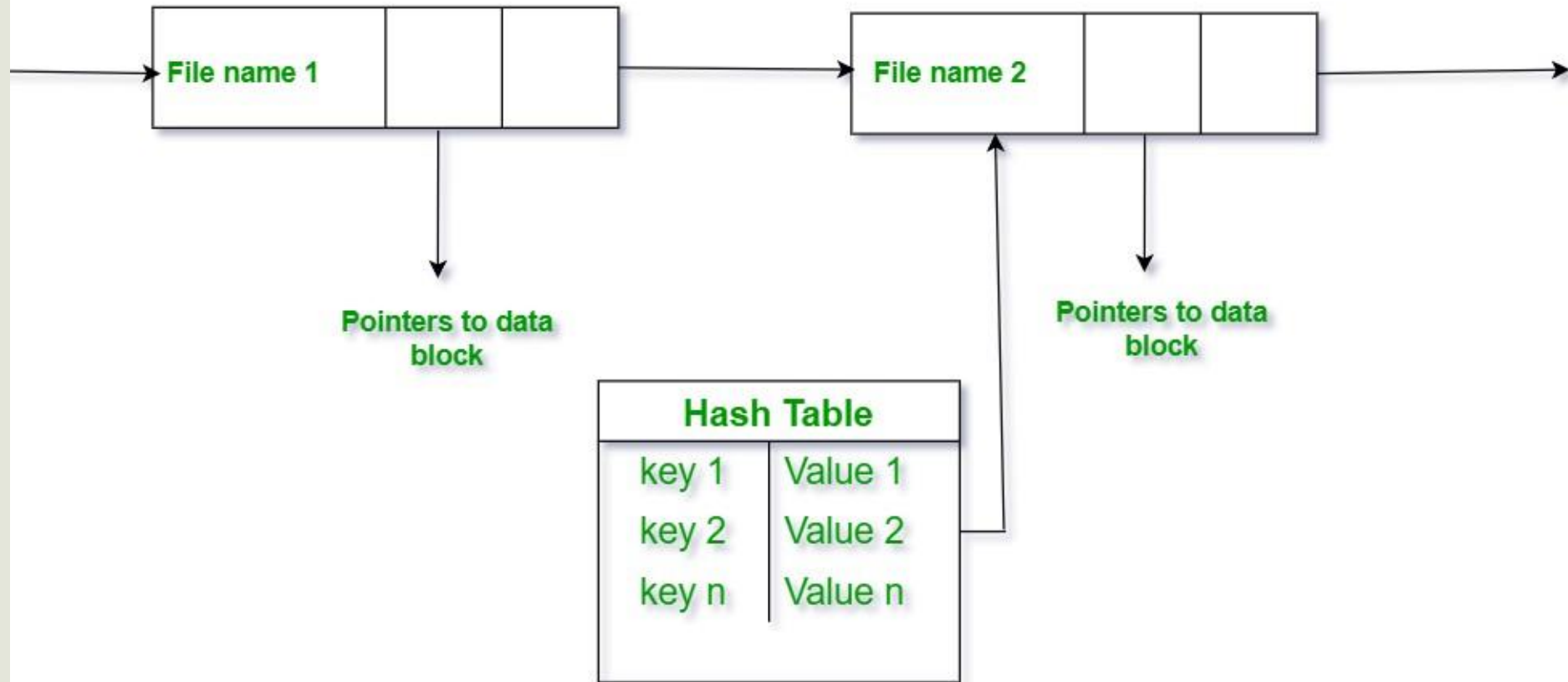
- To reduce search time
 - Operating systems implement a **software cache** to store the most recently used directory information.
 - Maintaining a **sorted list** also allows a binary search and reduces the search time.
- But maintaining a sorted list is difficult. When new entries are added, the new entries should be added to the appropriate position.

Directory Implementation – Hash

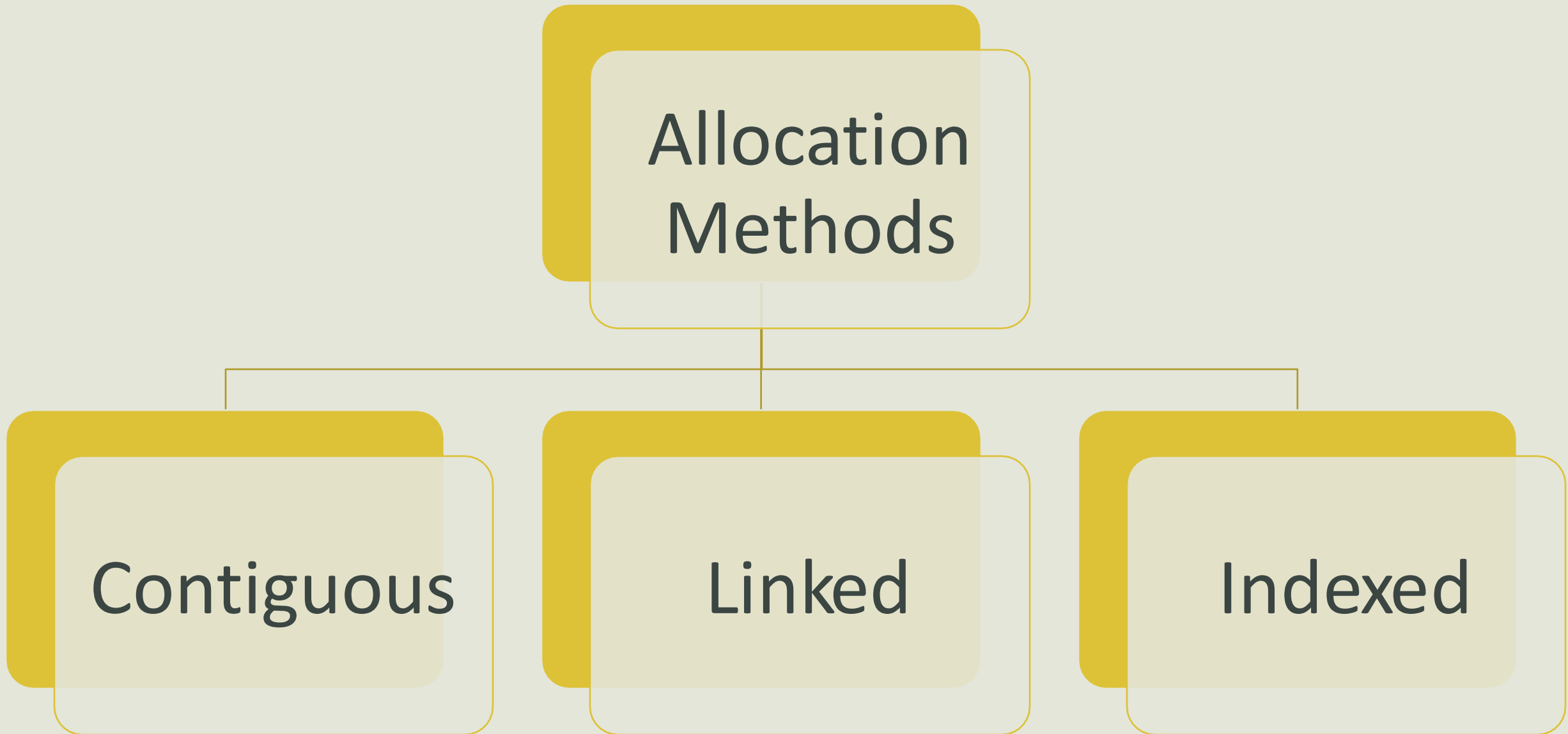
- Name and returns a pointer to the file name in the linear list
- A linear list stores the directory entries, but a hash data structure is also used
- The hash table takes a value computed from the file name and returns a pointer to the file name in the linear list.
- Fixed size and the dependence of the hash function on that size

Directory Implementation – Hash

Directory Implementation Using Hash Table



File System Implementation – Allocation Methods



File System Implementation – Allocation Methods

- Allocation methods deals with
 - how to allocate space to the files so that disk space is utilized effectively and
 - files can be accessed quickly.

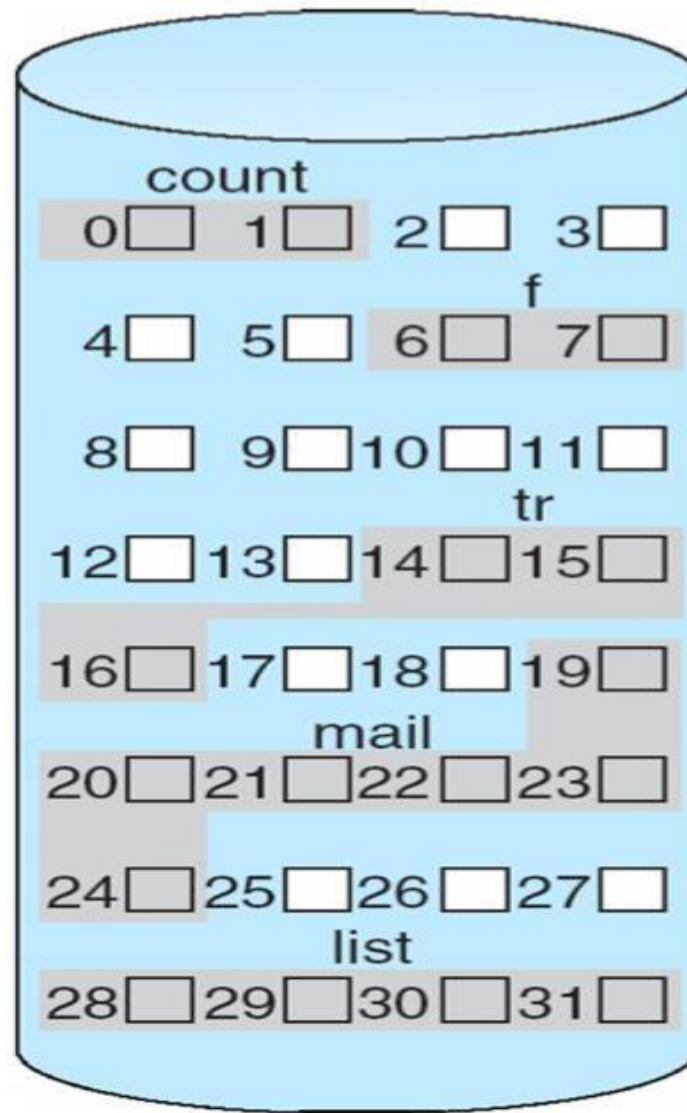
File System Implementation – Allocation Methods

- **Contiguous Allocation**

- Requires **each file** to **occupy** a set of **contiguous blocks** on the disk.
- **Contiguous allocation of a file** is defined by the **disk address and length** (in block units) of the first block.
 - If the file is **n blocks long** and **starts at location b**, then it occupies blocks **b, b+1, b+2, ..., b+n-1**
- The **directory entry** for **each file** indicates the **address of the starting block** and the **length allocated** for this file.

File System Implementation – Allocation Methods

▪ Contiguous Allocation



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

File System Implementation – Allocation Methods

▪ Contiguous Allocation - Demerits

- It suffers from the problem of external fragmentation
- When the file is created, the total amount of space it will need must be specified. It is not possible for all files
- Even if the total amount of space needed for a file is known in advance pre-allocation may be inefficient
- A file that grows slowly over a long period (months or years) must be allocated enough space for its final size therefore has an internal fragmentation

File System Implementation – Allocation Methods

- **Contiguous Allocation - Demerits**

- To overcome these disadvantages:

- Use a **modified contiguous** allocation scheme, in which a **contiguous chunk of space** called as an **extent** is allocated initially and then,
 - when that amount is not large enough **another chunk of contiguous space** an **extent** is added to the initial allocation.

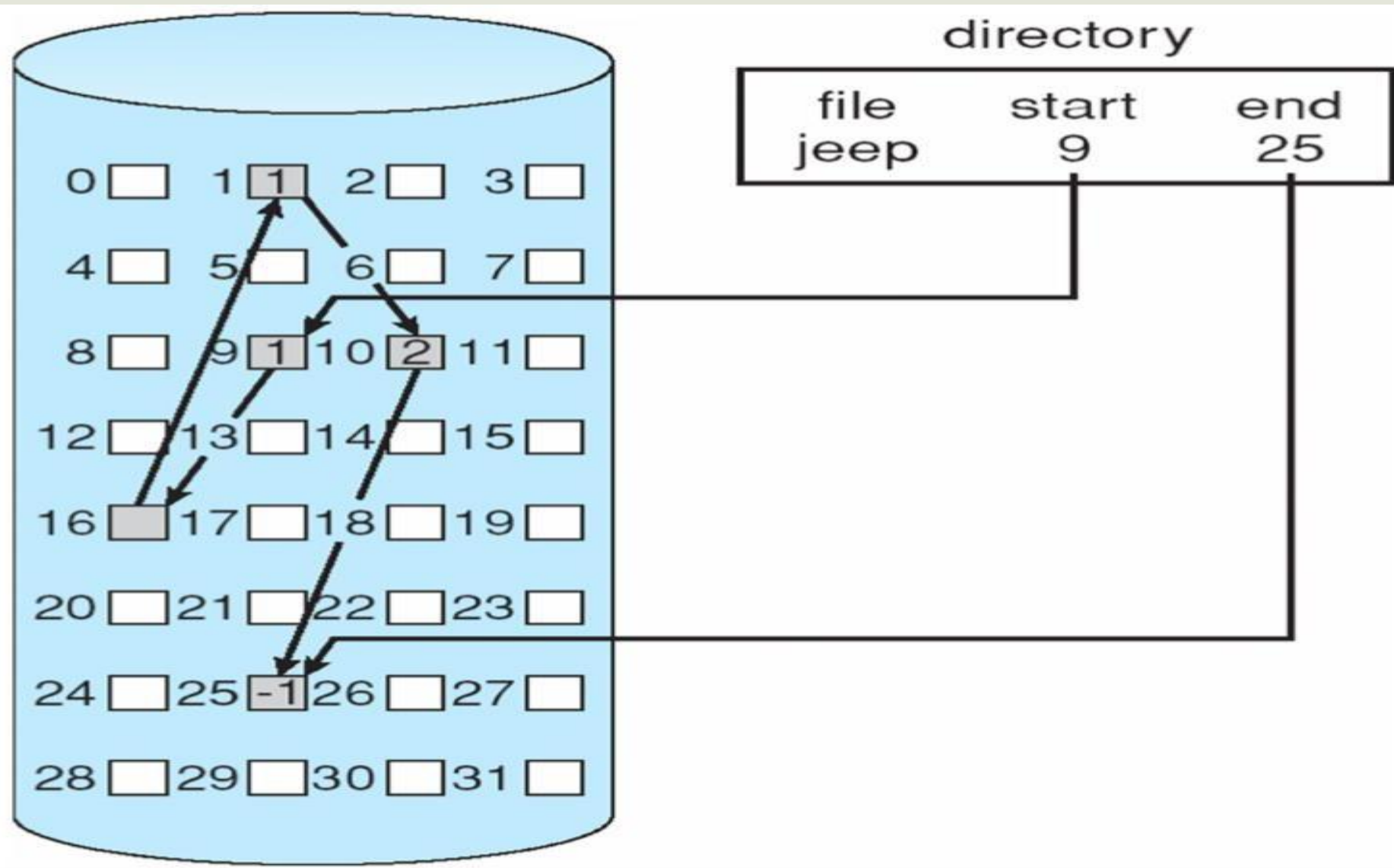
File System Implementation – Allocation Methods

- **Linked Allocation**

- Each **file is a linked list of disk blocks**, the disk blocks may be scattered anywhere on the disk
- The **directory contains a pointer to the first and last blocks of the file**.
 - For example, a file of five blocks might start at block 9, continue at block 16, then block 1, block 10, and finally block 25.
- Each block contains a pointer to the next block. These pointers are not made available to the user.
- There is no external fragmentation with linked allocation, and any free block on the free space list can be used to satisfy a request.
- The size of a file does not need to be declared when that file is created

File System Implementation – Allocation Methods

▪ Linked Allocation



File System Implementation – Allocation Methods

▪ Linked Allocation - Demerits

- To find the i^{th} block of a file, start at the beginning of that file, and follow the pointers until get to the i^{th} block.
- It is inefficient to support a direct access capability for linked allocation files.
- If a pointer requires 4 bytes out of a 512-byte block, then 0.78 percent of the disk is being used for pointers, rather than for information
- Since the files are linked together by pointers scattered all over the disk hardware failure might result in picking up the wrong pointer

File System Implementation – Allocation Methods

- **Linked Allocation - Solution to 2nd demerit**
 - Collect blocks into multiples, called clusters, and to allocate the clusters rather than blocks.

File System Implementation – Allocation Methods

- **Linked Allocation - File Allocation Table**
 - An important variation on the linked allocation method is the use of a file allocation table(FAT).
 - This simple but efficient method of disk-space allocation is used by the MS-DOS and OS/2 operating systems.
 - A section of disk at beginning of each partition is set aside to contain the table.
 - The table has entry for each disk block and is indexed by block number.
 - The FAT is much as is a linked list.

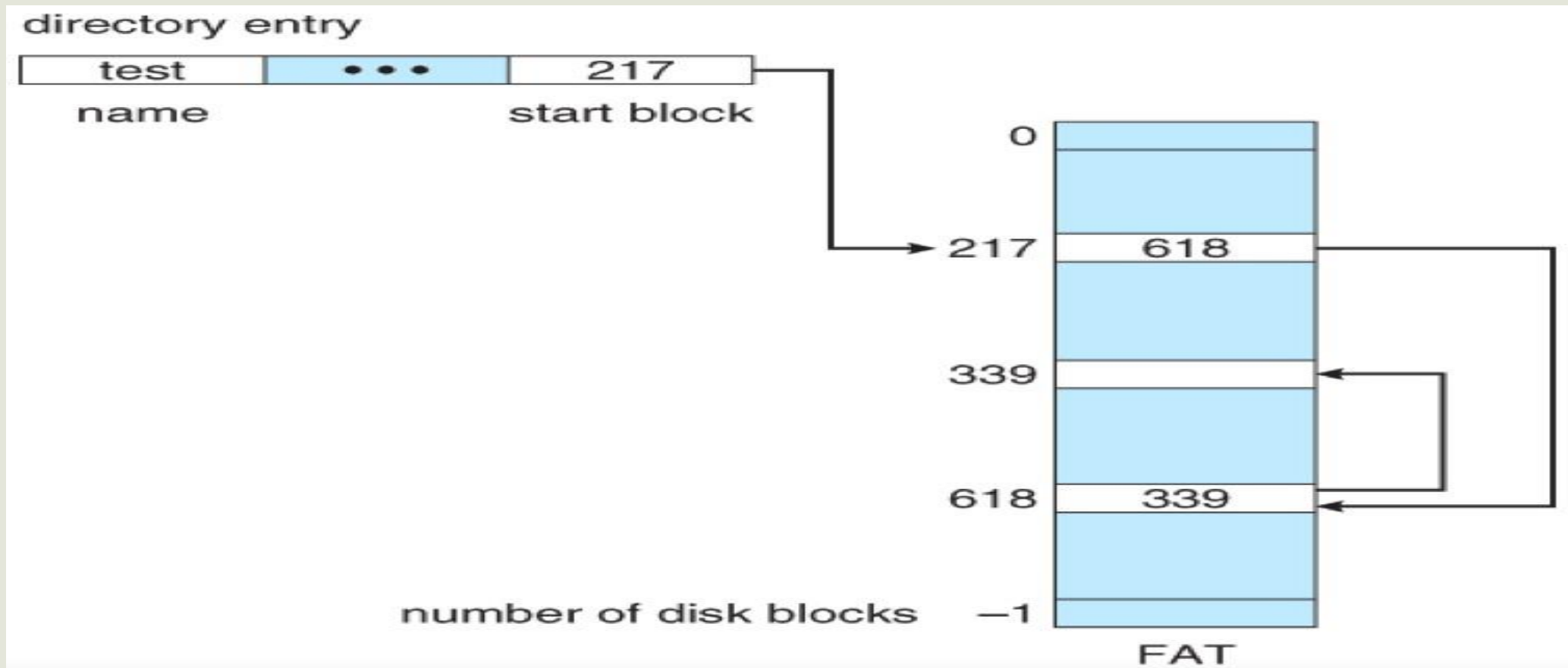
File System Implementation – Allocation Methods

- **Linked Allocation - File Allocation Table**
 - The **directory entry** contains the **starting block number of the file**.
 - The table entry indexed by that **block number** contains the **block number of the next block in the file**.
 - This **chain continues until the last block** which has a **special end – of – file value** as the table entry.
 - Unused blocks are indicated by a 0-table value.
 - Allocating a new block file is a simple matter of finding the first 0 – valued table entry and replacing the previous end of file value with the address of the new block.

File System Implementation – Allocation Methods

- **Linked Allocation - File Allocation Table**

- Example- the FAT structure for a file consisting of disk blocks 217, 618, and 339.



File System Implementation – Allocation Methods

- Indexed Allocation

- **Linked allocation** solves the **external – fragmentation** and **size- declaration problems** of contiguous allocation.
- Linked allocation **cannot support efficient direct access**, since the pointers to the blocks are scattered.
- Indexed allocation solves this problem by bringing all the pointers together into **one location: the index block**.

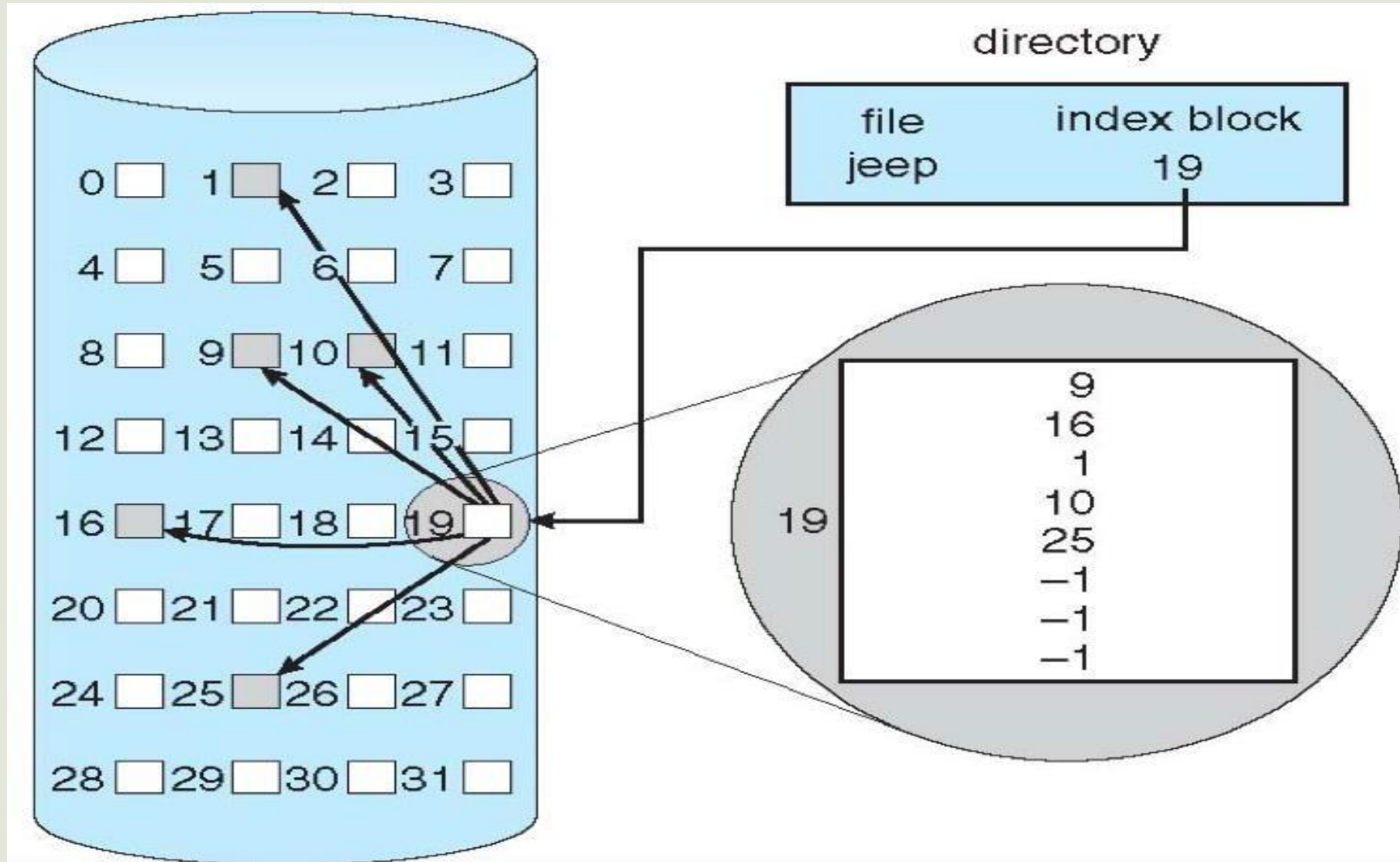
File System Implementation – Allocation Methods

▪ Indexed Allocation

- Each file has its own index block, which is an array of disk – block addresses.
- The i^{th} entry in the index block points to the i^{th} block of the file.
- The directory contains the address of the index block .
- To read the i^{th} block, we use the pointer in the i^{th} index –block entry to find and read the desired block this scheme is like the paging scheme.

File System Implementation – Allocation Methods

▪ Indexed Allocation



File System Implementation – Allocation Methods

- **Indexed Allocation - Demerits**

- Indexed allocation suffers from wasted space. The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.
- If the index block is too small, however, it will not be able to hold enough pointers for a large file.
- **Linked Scheme:** An index block is normally one disk block. Thus, it can be read and written directly by itself. To allow for large files, we may link together several index blocks.
- **Multilevel index:** A variant of the linked representation is to use a first level index block to point to a set of second-level index blocks

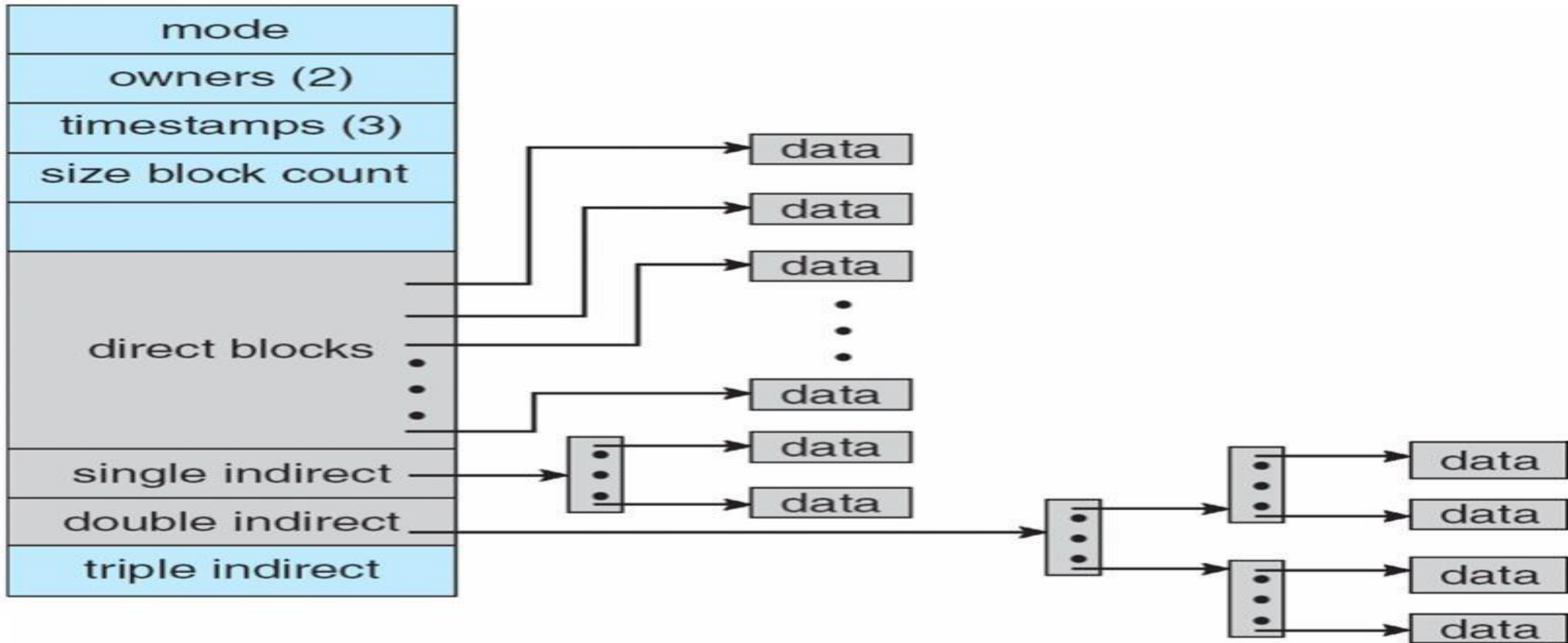
File System Implementation – Allocation Methods

- **Indexed Allocation - Combined Scheme**

- Another alternative, used in the UFS, is to keep the first, say, 15 pointers of the index block in the file's inode.
- The first 12 of these pointers point to direct blocks; that is for small (no more than 12 blocks) files do not need a separate index block
- The next pointer is the address of a single indirect block.
- The single indirect block is an index block, containing not data, but rather the addresses of blocks that do contain data.
- Then there is a double indirect block pointer, which contains the address of a block that contains pointers to the actual data blocks. The last pointer would contain pointers to the actual data blocks.
- The last pointer would contain the address of a triple indirect block.

File System Implementation – Allocation Methods

▪ Indexed Allocation - Combined Scheme



File System Implementation – Allocation Methods

- Best allocation method depends on file access type
 - **Contiguous** is great for **sequential and random**
 - **Linked** is good for **sequential, not random**
 - **Indexed** (combined) is more **complex**
 - Single block access may require 2 index block reads then data block read
 - Clustering can help improve throughput, reduce CPU overhead
 - Cluster is a set of contiguous blocks

File System Implementation – Allocation Methods

- Disk I/O is slow, reduce as many disk I/Os as possible
 - Intel Core i7 extreme edition 990x (2011) at 3.46Ghz = 159,000 MIPS
 - typical disk drive at 250 I/Os per second
 - $159,000 \text{ MIPS} / 250 = 630$ million instructions during one disk I/O
 - Fast SSD drives provide 60,000 IOPS
 - $159,000 \text{ MIPS} / 60,000 = 2.65$ millions instructions during one disk I/O